

Program translation using model-driven engineering

K. Lano

kevin.lano@kcl.ac.uk

Dept. of Informatics, King's College London
London, UK

ABSTRACT

The porting or translation of software applications from one programming language to another is a common requirement of organisations that utilise software, and the increasing number and diversity of programming languages makes this capability as relevant today as in previous decades.

Several approaches have been used to address this challenge, including machine learning and the manual definition of explicit translation rules. We define a novel approach using model-driven engineering (MDE) techniques: reverse-engineering source programs into specifications in the UML and OCL formalisms, and then forward-engineering the specifications to the required target language. This approach has the additional advantage of extracting specifications of software from code. We provide an evaluation based on a comprehensive dataset of examples, including industrial cases, and compare our results to those of other approaches and tools.

ACM Reference Format:

K. Lano. 2022. Program translation using model-driven engineering. In *44th International Conference on Software Engineering Companion (ICSE '22 Companion)*, May 21–29, 2022, Pittsburgh, PA, USA. ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/3510454.3528639>

1 INTRODUCTION

Over 100 programming languages are listed on the TIOBE index¹, and new languages are introduced on a regular basis. This leads to a significant demand from organisations to modernise or migrate their software assets by translating these from one programming language or version to another.

We divide the translation process into two steps: (i) abstraction of the source program into a specification, expressed in the UML and OCL international standard software modelling languages [14]; (ii) forward engineering of the specification into the target language. This approach takes advantage of the many MDE tools which provide forward engineering from UML/OCL into multiple target languages, for example [6, 15]. Hence we can focus effort on the reverse-engineering step and the representation of programming concepts and constructs using UML and OCL.

¹<https://www.tiobe.com/tiobe-index>

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
ICSE '22 Companion, May 21–29, 2022, Pittsburgh, PA, USA
© 2022 Copyright held by the owner/author(s).
ACM ISBN 978-1-6654-9598-1/22/05.
<https://doi.org/10.1145/3510454.3528639>

2 MDE PROCESS FOR LANGUAGE TRANSLATION

The translation process consists of successive steps of parsing, abstraction and forward engineering (Figure 1).

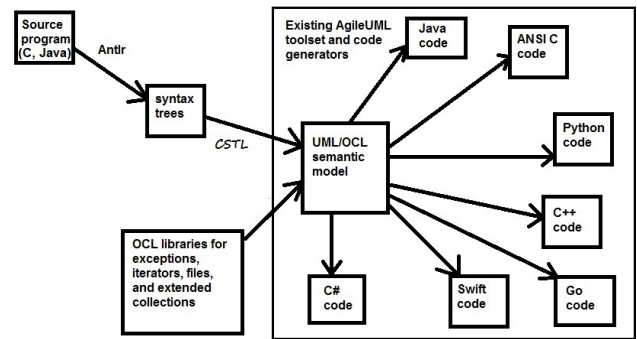


Figure 1: MDE program translation process

In order to translate programs of source language $\mathcal{L}_1$ to programs of target language $\mathcal{L}_2$, a parser for $\mathcal{L}_1$ is used to produce parse trees, which are then input to an abstraction transformation for the language, written using *CSTL* [13]. The output is a UML/OCL specification in textual form, consisting of class specifications with data features and operations. We provide additional OCL libraries to represent program semantics for aspects such as files, reflection, dates, exceptions and iterators, which are not present in standard OCL. Forward engineering using AgileUML [6] is then used to map the abstracted specification to the target language.

2.1 Program abstraction

We use Antlr [2] to parse program texts, and use the *CSTL* text-to-text transformation language [13] to map program language parse trees to a text form of UML/OCL. The advantage of *CSTL* is that the mapping can be easily modified or extended by end-users.

Examples of the abstraction rules for Java expressions are:

```
expression::
  _1 ? _2 : _3 |--> if _1 then _2 else _3 endif

( _1 ) _2 |--> _2->oclAsType(_1)

_1 instanceof _2 |--> _1->oclIsKindOf(_2)
```

The LHS of a rule matches against $\mathcal{L}_1$ parse tree nodes with the same structure and concrete tokens. Other subterms are assigned to variables $_i$ in order, and recursively mapped to $\mathcal{L}_2$ text.

To define program executable behaviour, we adopt the AgileUML textual notation for UML structured activities [6]. This is similar to the extended OCL of [4]. In the abstraction of statements, we remove unstructured control flow elements, such as C's *goto* and *switch*, using the *continuation* concept [16] to replace *goto* labels by new functions. We do not currently translate GUI facilities. For language subsets (numeric types and computations without overflow, underflow or exceptions; data structures used with value semantics) a formally correct abstraction mapping can be defined.

3 EVALUATION AND COMPARISON

Abstraction mappings to UML/OCL from Java 6/7 and from ANSI C have been developed. These consist of 1940 and 2160 LOC and took 4 person months and 2.5 person months of effort to develop, respectively. For Java 6/7, 73% of the grammar rules have corresponding abstraction rules. For C, 90% of the grammar rules of [9] are covered, and 158 of 179 library operations/fields (88%).

To evaluate the correctness of the translations, we run a series of tests on both source and target versions of translation cases, and compute the percentage of test results which agree. This measure of accuracy is *computational accuracy*, also used in [10].

For the Java 6/7 to Python, Swift, C#, C++ and Go via UML/OCL translations, we used 100 evaluation cases, including three cases taken from a package of financial software. Table 1 summarises the cases and results.

Category	Cases (Tests)	Translation accuracy				
		Python	Swift	C#	Go	C++
java.io	9 (31)	100%	92%	91%	91%	93%
java.lang	25 (123)	89%	95%	96%	81%	95%
java.math	3 (18)	100%	95%	100%	95%	90%
java.util	24 (119)	97%	100%	98%	99%	97%
Language features	34 (102)	88%	91%	91%	86%	83%
Apps	5 (46)	100%	100%	96%	98%	100%
Total	100 (439)	93%	96%	96%	90%	93%

Table 1: Translation from Java 6/7 cases

These results can be compared with those of java2python and Transcoder for the Java to Python cases reported in [10]. The cases of [10] are also program fragments and small-scale programs similar in size and complexity to our examples. Java2python only achieves an accuracy of 38.3% on the Java to Python examples of [10], whilst Transcoder achieves 68.7% accuracy. Our results (using a similar number of tests – 463 tests are used in [10]) are significantly better than these scores. Table 2 gives the corresponding results for C translation cases. There are 70 C cases.

The evaluation dataset is at zenodo.org/record/6386382. The script *java2py3* combines abstraction and forward engineering steps to provide a seamless translation from Java source files to Python.

4 RELATED WORK

Two general strategies have been used for program translation: (i) via heuristic manually-created rules [8, 17]; (ii) machine learning

Category	Cases (Tests)	Translation accuracy		
		Swift	C#	Go
Statements	14 (33)	97%	100%	100%
Declarations/ Types	36 (53)	72%	79%	93%
Libraries	20 (61)	88%	96%	85%
Total	70 (147)	84%	90%	91%

Table 2: Translation from ANSI C cases

approaches inducing implicit rules from corresponding examples in the source and target languages [1, 5, 7]. The first strategy has the problem that a large number of specific rules are necessary to translate from one programming language to another, and the effort must be repeated for each pair of languages under consideration. The second approach requires large datasets of program examples [7, 10].

The MoDisco MDE approach for reverse engineering uses meta-models to represent the structure and syntax of source language programs [3]. Unlike our approach, this does not provide fine-grained semantic representations of source program semantics. Internal intermediate languages are an established means of factoring compilation or reverse-engineering processes, eg., [11]. In our approach, the UML/OCL application abstraction is intended as a human-readable specification, to support application analysis and evolution.

5 CONCLUSIONS AND FUTURE WORK

We have shown that a program translation approach based upon the use of MDE can be effective in supporting practical program translation tasks. Future work includes using symbolic machine learning to automate the construction of abstraction mappings [12].

REFERENCES

- [1] K. Aggarwal, M. Salameh, and A. Hindle. 2015. Using machine translation for converting Python 2 to Python 3 code. *PeerJ Preprints* (2015).
- [2] Antlr. 2021. <https://wwwantlr.org>.
- [3] H. Bruneliere, J. Cabot, G. Dupe, and F. Madiot. 2014. MoDisco: a model-driven reverse engineering framework. *IST 56* (2014), 1012–1032. Issue 8.
- [4] F. Buttner and M. Gogolla. 2014. On OCL-based imperative languages. *Science of Computer Programming* 92 (2014), 162–178.
- [5] X. Chen, C. Liu, and D. Song. 2018. Tree-to-tree neural networks for program translation. In *NIPS 2018*.
- [6] Eclipse Agile UML project. 2022. <https://projects.eclipse.org/projects/modeling.agileuml>, accessed 7.3.2022.
- [7] Facebook Research. 2021. TransCoder, github.com/facebookresearch/TransCoder.
- [8] Java2Python. [n. d.]. Java2Python, <https://github.com/natural/java2python>, 2021.
- [9] B. Kernighan and D. Ritchie. 1988. *The C Programming Language* (2nd ed.). Prentice Hall.
- [10] M-A. Lachaux, B. Roziere, L. Chanussot, and G. Lample. 2020. Unsupervised translation of programming languages. arXiv:2006.03511v3.
- [11] K. Lano and P. Breuer. 1990. From Programs to Z Specifications. In *Z User Workshop*. Springer London, 46–70.
- [12] K. Lano and Q. Xue. 2022. Code Generation by Example. In *Modelsworld 2022*.
- [13] K. Lano, Q. Xue, and S. Kolahdouz-Rahimi. 2020. Agile specification of code generators for model-driven engineering. In *ICSEA 2020*.
- [14] OMG. 2014. Object Constraint Language (OCL) 2.4 Specification. OMG document formal/2014-02-03.
- [15] Papyrus. 2020. Papyrus toolset, <https://www.eclipse.org/papyrus>.
- [16] J. Reynolds. 1993. The discoveries of continuations. *LISP and Symbolic Computation* 6 (1993), 233–247.
- [17] Tangible. 2021. <https://www.tangiblesoftwareolutions.com/converters.html>.